

列表生成式基础用法介绍

- 传统方法实现需求：要生成1到5所有数字的平方组成的列表，传统方法需要先定义空列表，再用for循环遍历1到5，依次计算平方后用append方法添加到列表，总共需要4行代码。
- 列表生成式简化写法：使用Python特有的列表生成式，可以用1行代码实现相同需求，写法为在列表中先写计算表达式，再写for循环遍历逻辑。
- 带条件过滤的列表生成式：如果需要筛选满足条件的结果，可以在列表生成式末尾添加if条件判断，只保留符合条件的计算结果进入最终列表。

```
In [12]: squares = []
for i in range(1, 6):
    if i**2 % 2 == 0:
        squares.append(i**2)
print(squares)
squares = [i**2 for i in range(1, 10)]
print(squares)
```

```
[4, 16]
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81]
```

元组基础定义与声明规则

- 元组与列表的核心区别：元组和列表结构相似，核心区别是元组的元素不能修改；列表使用中括号声明，元组使用小括号声明。
- 空元组的使用说明：虽然可以声明空元组，但实际开发中不推荐声明空元组，因为元组无法修改，空元组没有实际使用意义。
- 单元素元组声明规则：声明只包含1个元素的元组时，必须在元素后添加逗号，否则Python会将小括号识别为优先级提升符号，将内容识别为原本的数据类型。
- 类型转换生成元组：可以通过tuple()函数将列表等其他数据类型转换为元组，也能生成长度为1的元组。

元组支持的操作说明

- 允许的操作类型：元组只支持所有查询类操作，包括根据索引获取元素、查询元素索引、统计元素出现次数、计算长度、切片遍历，操作方法和列表完全一致。
- 禁止的操作类型：元组不支持新增、删除、修改、排序、反转等会改变元素内容的操作，强行修改会触发程序报错。
- 强制修改元组的方法：如果确实需要修改元组内容，可以先将元组转换为列表，修改完成后再重新转换回元组，将新结果赋值给原变量。

元组特殊用法与应用场景

- 多变量批量赋值：可以将元组中的元素依次赋值给多个变量，要求左侧变量的个数必须和元组元素的个数完全一致，否则会报错，该写法常用于接收函数的多个返回值。
- 保护数据安全性：在数据传递过程中，如果不希望接收方修改数据，可以将数据封装为元组类型，通过不可修改的特性保护数据权限。
- 格式化字符串参数：Python格式化字符串使用百分号传参时，小括号包裹的多参数就是元组类型。

- 函数可变参数实现：Python函数的可变长度参数，会将传入的多个参数打包为元组，print函数支持输出任意多个内容，就是基于元组实现的。

```
In [ ]: names = ('张三', '李四', '王五', '赵六')
for name in names:
    print(name)
print(type(names))
print(names[0])
print(names[1:3])
print(names[-1])
print(names.count('张三'))
peirs = [('张三', 20), ('李四', 30), ('王五', 40)]
for name, age in peirs:
    print(f"{name}的年龄是{age}")
print(tuple([10, 20, 30]))
print(list([10, 20, 30]))
#使用for in遍历列表
```

```
张三
李四
王五
赵六
<class 'tuple'>
张三
('李四', '王五')
赵六
1
张三的年龄是20
李四的年龄是30
王五的年龄是40
(10, 20, 30)
[10, 20, 30]
```

```
In [32]: s = 'hello,world'
print(s[0])
print(s[-1]) #切片
print(s[1:5])
#for循环
for idx in range(len(s)):
    print(f"索引 {idx}: {s[idx]}")

#while循环
idx = 0
while idx < len(s):
    print(f"索引 {idx}: {s[idx]}")
    idx += 1

for c in s:
    print(c, end=' ')
```

```

h
d
ello
索引 0: h
索引 1: e
索引 2: l
索引 3: l
索引 4: o
索引 5: ,
索引 6: w
索引 7: o
索引 8: r
索引 9: l
索引 10: d
索引 0: h
索引 1: e
索引 2: l
索引 3: l
索引 4: o
索引 5: ,
索引 6: w
索引 7: o
索引 8: r
索引 9: l
索引 10: d
h e l l o , w o r l d

```

字符串索引取值操作

- 正负数索引规则：字符串的每个字符都对应一个从0开始的正向索引，从末尾开始计数的索引为负数索引，末尾第一个字符对应索引-1。
- 示例验证：以诗句“床前明月光”为例，取第一个字符使用索引 `[0]`，取末尾最后一个字符“光”使用索引 `[-1]`，两种取值方式都可得到正确结果。

字符串遍历方法

- for直接遍历：直接使用 `for 字符 in 字符串` 的语法，可以依次取出字符串中的每个字符，配合 `end=' '` 可以用空格分隔输出每个字符。
- for索引遍历：通过 `for idx in range(len(字符串))` 生成索引范围，再通过 `字符串[idx]` 取出对应字符，可同时得到每个字符的正向索引。
- 同时获取索引与字符：使用 `for idx, c in enumerate(字符串)` 语法，可以一次性同时输出每个字符对应的索引和字符本身，该写法实际开发中使用频率更高。
- while循环遍历：while循环只能通过索引遍历，需要先定义初始索引 `idx=0`，循环条件为 `idx < len(字符串)`，每次循环取出字符后将 `idx` 自增1，即可遍历所有字符。

字符串切片操作

- 基础切片规则：切片语法为 `字符串[起始索引:结束索引:步长]`，结果不包含结束索引对应的字符，默认起始索引为0，默认结束索引为字符串长度，默认步长为1。
- 指定切片示例：要从“床前明月光”中取出“明月”两个字，可使用切片 `[2:4]`，“明”对应索引2，“月”对应索引3，结束索引写4即可正确切出目标内容。
- 步长的特殊用法：步长设置为-1时代表反向取值，会将字符串倒序输出，“床前明月光”使用步长-1切片后会输出“光月明前床”。步长设置为2时会跳着取值，取出索引为

0、2、4的对应字符。

- 正负索引切片规则：正数步长从左向右取值，起始索引在左、结束索引在右；负数步长从右向左取值，起始索引在右、结束索引在左，和列表、元组的切片规则完全一致。

字符串常用函数分类说明

- 不可变特性：字符串本身是不可变类型，无法直接修改原字符串，所有字符串操作都会生成新的字符串。
- 判断类函数：判断类函数主要用来判断字符串的内容类型，包括判断是否全为字母、是否全为数字、是否包含字母或数字、是否为空格等，中文开发场景中大小写相关判断使用频率极低。
- 查找替换类函数：常用函数包括判断字符串开头/结尾、查找子串索引的 `index` 函数、替换子串的 `replace` 函数、统计子串出现次数的 `count` 函数，其中 `replace` 使用频率较高。
- 拆分连接类函数：拆分字符串的 `split` 函数使用频率很高，可以按照指定分隔符将字符串切分为多个子串；`join` 函数可以用指定间隔符将列表等可迭代对象合并为一个字符串；`strip` 函数可以去除字符串两端的空白字符。

```
In [36]: str1 = '123 一 二 叁 肆 伍 陆 柒 捌 玖'
print("123".isdecimal())
print("123 一 二 叁".isdigit())
print(str1.isnumeric())
```

```
True
False
False
```

```
In [54]: #查找和替换
print("01.txt".endswith('.txt'))
print('abcdefg'.find('c'))
print('abcdefg'.find('x'))
```

```
True
2
-1
```

```
In [ ]: str1 = "zhangsan lisi wangwu zhaoliu"
names = str1.split(",")
print(names)
print(names[0])
print("#".join(names))#字符串的连接
#字符串的切片
path = "D:/project/step1/week2/0525.ipynb"
arr = path.split("/")
print(arr)
print(arr[-1])
```

```
['zhangsan lisi wangwu zhaoliu']
zhangsan lisi wangwu zhaoliu
zhangsan lisi wangwu zhaoliu
['D:', 'project', 'step1', 'week2', '0525.ipynb']
0525.ipynb
```

字典dict，映射关系 `d={key1:val1,key2:val2}`

```
In [ ]: #字典
dict1 = {"name": "zhangsan", "age": 18, "sex": "男"}
dict2 = {"name": "lisi", "age": 20, "sex": "女"}
#申明字典
dict3 = {}
print(type(dict3))
#添加键值对
dict3["name"] = "wangwu"
dict3["age"] = 25
dict3["sex"] = "男"
print(dict3)
dict2.setdefault("hobby", "打瓦")#如果键不存在则添加键值对, 如果键存在则不修改原有
print(dict2)
#合并字典
dict2.update(dict3)
print(dict2)
#删除键值对
del dict2["hobby"]
print(dict2)
print(dict2.get("name"))#获取键对应的值, 如果键不存在则返回None
print(dict2.get("hobby", "没有这个键"))#获取键对应的值, 如果键不存在则返回默认值
print(dict2.keys())#获取字典的所有键
print(dict2.values())#获取字典的所有值
print(dict2.items())#获取字典的所有键值对
print(len(dict2))#获取字典的长度
#遍历字典
for key in dict2:
    print(f"{key}: {dict2[key]}")
#判断键是否在字典中
print("name" in dict2)
print("hobby" in dict2)
#遍历键值对
for key, value in dict2.items():
    print(f"{key}: {value}")
```

```
<class 'dict'>
{'name': 'wangwu', 'age': 25, 'sex': '男'}
{'name': 'lisi', 'age': 20, 'sex': '女', 'hobby': '打瓦'}
{'name': 'wangwu', 'age': 25, 'sex': '男', 'hobby': '打瓦'}
{'name': 'wangwu', 'age': 25, 'sex': '男'}
wangwu
没有这个键
dict_keys(['name', 'age', 'sex'])
dict_values(['wangwu', 25, '男'])
dict_items([('name', 'wangwu'), ('age', 25), ('sex', '男')])
3
name: wangwu
age: 25
sex: 男
True
False
name: wangwu
age: 25
sex: 男
```

Python基础内置数据类型分类

- 核心类型梳理: Python基础阶段需要掌握7种常用内置数据类型, 分别是整数int、浮点数float、布尔值bool、字符串str、列表list、元组tuple、字典dict、集合set。后

续还会接触自定义class创建的自定义类型。

- 基础数据类型特点：布尔类型只有True和False两个值，数字0、空字符串、空列表、空元组转换为布尔类型为False，其余内容转换结果为True。字符串由单引号或双引号包裹，属于不可变序列，支持切片、循环、判断、分割、替换等操作。
- 列表与元组的差异：列表使用中括号声明，是有序可修改的序列，支持增删改查等多种操作。元组使用小括号声明，是不可修改的有序序列，仅支持查询、切片和遍历操作。

字典类型的应用场景与定义

- 场景优势对比：列表适合存储有序同类型数据，存储带映射关系的个人信息（姓名、年龄、性别）时，必须严格按顺序存储，错位会导致语义错误，且无法直观体现语义关联。字典以键值对存储映射关系，只要键值对应正确，存储顺序不影响结果，可读性更强。
- 字典的核心规则：字典以键值对形式存储数据，键（key）必须唯一且只能为不可变类型（字符串、数字、元组），实际开发中字符串作为键使用最多。值（value）不要求唯一，可以是任意数据类型。
- 字典的声明方式：字典使用花括号声明，一个字典包裹在一对花括号中，每个键值对内部用冒号分隔键和值，多个键值对之间用逗号分隔。声明时可以直接带初始键值对，也可以声明空字典，空字典同样属于dict类型。

字典的新增操作

- 基础中括号赋值法：使用 `字典变量[键] = 值` 的形式新增数据，若字典中不存在该键，则会新增一个对应的键值对；若字典中已经存在该键，则会修改该键对应的值。这是开发中最常用的新增修改方式。
- `setdefault`方法说明：`setdefault` 方法在键不存在时会新增键值对，键存在时只会返回该键对应的值，不会修改原有值，该方法在实际开发中使用较少。
- `update`方法说明：`update` 方法用于将两个字典合并，把传入字典的所有键值对添加到调用方法的字典中，该方法使用频率也较低。

字典的修改操作

- 核心修改规则：由于字典的键是唯一不可重复的，当使用 `字典变量[键] = 值` 赋值时，若键已经存在，会直接覆盖原有值，完成修改操作，键本身不会发生变化。
- `setdefault`无法做修改：调用 `setdefault` 方法时，如果传入的键已经存在，只会返回键对应的原有值，不会修改原有值，因此无法用该方法完成修改操作。

字典的删除操作

- `del`关键字删除：使用 `del 字典变量[键]` 可以删除指定键的键值对，如果要删除的键不存在，会抛出KeyError错误导致程序终止运行。
- `pop`方法删除：使用 `字典变量.pop(键)` 可以删除指定键的键值对，并返回该键对应的值，如果要删除的键不存在，同样会抛出KeyError错误。
- `clear`方法清空：使用 `字典变量.clear()` 会清空字典中所有的键值对，最终得到一个空字典，该方法在各类数据类型中用法一致。

不同数据类型的符号区分规则

- 符号识别总结：可以通过包裹符号快速区分不同的数据类型，其中字符串由单引号或双引号包裹，列表使用中括号，元组使用小括号，字典使用花括号。
- 符号分隔规则：所有容器类型（列表、元组、字典）的多个元素之间，都使用逗号进行分隔，仅字典的单个元素为键值对，需要额外用冒号分隔键和值。

Python字典get()获取值方法介绍

- 基础取值规则：使用 `get()` 方法可以替换中括号根据键获取字典对应的值，当键不存在时，默认返回 `None`，不会抛出程序错误。
- 自定义默认值规则：可以在 `get()` 方法中传入第二个参数，作为键不存在时的自定义返回默认值，该参数支持任意数据类型，包括整数、字符串等。
- 适用频率说明：该方法在后续Python开发中使用频率很高，是字典操作的核心常用方法。

获取字典键值对数量的方法

- 使用内置方法获取长度：可以通过 `len()` 函数传入字典对象，获取字典中包含的键值对总数量，该数值对应字典的长度。
- 示例验证结果：示例字典 `dict二` 中一共包含5个不同的键，运行 `len(dict二)` 后得到的返回结果为5，和实际的键值对数量匹配。
- 概念说明：字典中的每个存储单元都称为键值对，长度就是字典内部存储的键值对总个数。

获取字典所有键的keys()方法

- 方法调用格式：调用格式为 `字典对象.keys()`，该方法会提取字典中所有的键，返回一个 `dict_keys` 类型的结果。
- 返回结果特性：返回结果放在类似列表的中括号结构中，支持对其进行循环遍历，可以逐个取出每一个键。

通过keys()遍历字典的方法

- 遍历实现逻辑：通过for循环遍历 `字典.keys()` 取出所有键后，再利用 `字典[键]` 的方式获取对应键的值，即可输出全部键值对。
- 代码实现示例：示例代码为 `for key in dict二.keys(): val = dict二[key]; print(f"{key}={val}")`，运行后可以逐行输出字典中每一组键值对。
- 应用结论：该方法是遍历字典获取所有键值对的标准可行方法，逻辑清晰容易实现。

获取字典所有值的values()方法

- 方法调用格式：调用格式为 `字典对象.values()`，该方法会提取字典中所有的值，不保留对应键信息。
- 结果特性说明：字典中允许值重复，因此提取出的多个值可能完全相同，无法反向对应回原有的键。
- 适用场景说明：该方法使用频率非常低，一般业务中都是通过唯一的键查询对应的值，不需要单独提取所有值。

key存在性判断方法

- 判断语法规则：使用语法 `key in 字典对象` 进行判断，如果键存在于字典中就返回 `True`，不存在则返回 `False`。
- 实际应用场景：可以提前判断键是否存在，再执行查询或删除操作，避免程序抛出 `KeyError` 错误。
- 分支处理逻辑：判断后可以增加 `else` 分支，输出提示信息告知用户指定键不存在，提升程序的健壮性。

`items()`方法批量遍历键值对

- 方法调用格式：调用格式为 `字典对象.items()`，该方法会逐对取出字典中的每一组键值对。
- 遍历实现方式：在 `for` 循环中使用 `for key, val in 字典对象.items():` 的写法，两个变量分别接收每一组的键和值，不需要二次取值。
- 对比优势说明：和先取键再取值的方式相比，该方法代码更简洁，不需要根据键二次查询值，效率更高。

```
In [72]: #作业1
import random

shopList = [[], [], []]
goodsList = ['小米', '苹果', 'vivo', '华为', 'Oppo', 'Lenovo']

for goods in goodsList:
    random.choice(shopList).append(goods)

print(shopList)
```

```
[['vivo'], ['华为', 'Oppo', 'Lenovo'], ['小米', '苹果']]
```

```
In [73]: #作业2
digits = [1,2,3,4]
results = []

for i in digits:
    for j in digits:
        if j != i:
            for k in digits:
                if k != i and k != j:
                    results.append(100*i + 10*j + k)
print("总数量:", len(results))
print("所有的三位数:", results)
```

总数量: 24

所有的三位数: [123, 124, 132, 134, 142, 143, 213, 214, 231, 234, 241, 243, 312, 314, 321, 324, 341, 342, 412, 413, 421, 423, 431, 432]

```
In [74]: #作业3
name = " posekakaka "

# a
trimmed = name.strip()
print(trimmed)

# b
print(trimmed.startswith("po"))

# c
```



```

print(trimmed.endswith("a"))

# d
print(trimmed.replace("k", "c"))

# e
print(trimmed.split("k"))

# f
print(type(trimmed.split("k")))

```

posekakaka

True

True

posecacaca

['pose', 'a', 'a', 'a']

<class 'list'>

```

In [75]: #作业4
students = [
    {'name': 'Tom', 'age': 19, 'score': 92, 'sex': '女', 'tel': '15300022839'},
    {'name': 'Jerry', 'age': 20, 'score': 40, 'sex': '男', 'tel': '15300022838'},
    {'name': 'Andy', 'age': 18, 'score': 85, 'sex': '女', 'tel': '15300022837'},
    {'name': 'Jack', 'age': 16, 'score': 65, 'sex': '男', 'tel': '15300022428'},
    {'name': 'Rose', 'age': 17, 'score': 59, 'sex': '男', 'tel': '15300022653'},
    {'name': 'Bob', 'age': 18, 'score': 78, 'sex': '男', 'tel': '15300022867'}
]

# 3.1 遍历所有的姓名
for student in students:
    print(student['name'])

# 3.2 统计不及格学生的个数
fail_count = sum(1 for student in students if student['score'] < 60)
print("不及格人数:", fail_count)

# 3.3 打印所有男生的信息
for student in students:
    if student['sex'] == '男':
        print(student)

# 3.4 求平均分
average_score = sum(student['score'] for student in students) / len(students)
print("平均分:", average_score)

```

Tom

Jerry

Andy

Jack

Rose

Bob

不及格人数: 2

{ 'name': 'Jerry', 'age': 20, 'score': 40, 'sex': '男', 'tel': '15300022838' }

{ 'name': 'Jack', 'age': 16, 'score': 65, 'sex': '男', 'tel': '15300022428' }

{ 'name': 'Rose', 'age': 17, 'score': 59, 'sex': '男', 'tel': '15300022653' }

{ 'name': 'Bob', 'age': 18, 'score': 78, 'sex': '男', 'tel': '15300022867' }

平均分: 69.83333333333333

```

In [76]: #作业5
from collections import Counter

```

```
letters = 'abcdabcdabcdabcefg'
counts = Counter(letters)

for ch, cnt in counts.items():
    print(ch, cnt)
```

```
a 4
b 4
c 4
d 3
e 1
f 1
g 1
```

```
In [79]: #作业6
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]
target = 10

seen = set()
pairs = []

for num in nums:
    comp = target - num
    if comp in seen:
        pairs.append((comp, num))
        seen.add(num)

print("{ " + ", ".join(f"{a}:{b}" for a, b in pairs) + " }")
```

```
{4:6, 3:7, 2:8, 1:9}
```